



Gymnázium J. K. Tyla

Autonomní pipeline pro vývoj softwaru

Návrh a realizace systému DarkFactory

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.c

Vedoucí práce: Jméno vedoucího práce

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Práce se zabývá návrhem a realizací autonomního systému pro vývoj softwaru, který přebírá rutinní kroky vývojového procesu — od přijetí požadavku přes jeho interpretaci a naplánování až po vytvoření a ověření změny. Teoretická část shrnuje principy kontinuální integrace, řízení verzí a orchestrace jazykových modelů. V praktické části je popsán systém DarkFactory, jeho architektura a governance, a jsou vyhodnoceny výsledky jeho nasazení na reálných repozitářích.

Klíčová slova

autonomní systémy, softwarové inženýrství, kontinuální integrace, orchestrace agentů, jazykové modely

Annotation

This thesis deals with the design and implementation of an autonomous software engineering system that takes over the routine steps of the development process — from intake of a request through its interpretation and planning to the creation and verification of a change. The theoretical part summarises the principles of continuous integration, version control and the orchestration of language models. The practical part describes the DarkFactory system, its architecture and governance, and evaluates the results of its deployment on real repositories.

Keywords

autonomous systems, software engineering, continuous integration, agent orchestration, language models

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	3
Annotation	3
Keywords	3
1 Úvod	7
1.1 Motivace	7
1.2 Cíl práce	7
1.3 Metodika	7
1.4 Struktura práce	8
2 Teoretická část	9
2.1 Řízení verzí	9
2.1.1 Větvě a jejich role	9
2.1.2 Model pull requestu	9
2.2 Kontinuální integrace	9
2.2.1 Požadované kontroly	9
2.2.2 Sestavení a artefakty	10
2.3 Jazykové modely a jejich orchestrace	10
2.3.1 Kontext a nástroje	10
2.3.2 Ověřitelnost výstupu	10
2.3.3 Problém jediného poskytovatele	10
2.4 Governance automatizovaných změn	10
2.4.1 Schvalovací body	10
2.4.2 Dohledatelnost	11
2.4.3 Hlášení selhání	11
3 Praktická část	12
3.1 Cíl a rozsah systému	12
3.2 Architektura	12
3.2.1 Sdílení místo kopírování	12
3.2.2 Jediný konfigurační soubor	12
3.3 Životní cyklus požadavku	12
3.4 Popis prostředí repozitáře	13
3.4.1 Domény a prostředí	13
3.4.2 Deklarace jako doplněk rozpoznávání	13

3.5	Orchestrace napříč poskytovateli	13
3.6	Ověřování změn	13
3.7	Hlášení selhání	14
4	Výsledky a diskuse	15
4.1	Metodika ověření	15
4.2	Sjednocení pracovních postupů	15
4.3	Rozšíření na texty	15
4.4	Chyby, které se projevily až v provozu	15
4.5	Diskuse	16
4.6	Omezení	16
5	Závěr	17
	Seznam příloh	18
A	Obsah přiloženého média	19
B	Struktura konfiguračního souboru	20
	Seznam zdrojů	21

Seznam obrázků a tabulek

Tabulka 1 Rozpoznávání prostředí podle souboru popisujícího balíček.	13
Tabulka 2 Počet vlastních skriptů před sjednocením a po něm.	15

1 Úvod

Vývoj softwaru se za posledních dvacet let z velké části zautomatizoval. Sestavení programu, spuštění testů, kontrola stylu i nasazení do provozu dnes obstarávají stroje a nikdo je nepovažuje za práci hodnou lidského času. Jeden krok však zůstal ruční: samotná změna zdrojového kódu. Právě tam se tvoří většina prodlev — mezi okamžikem, kdy někdo popíše požadavek, a okamžikem, kdy se změna dostane k uživatelům.

Výrobní průmysl zná pojem *dark factory*, tedy „temná továrna“: provoz, který běží bez lidské obsluhy, a proto v něm nemusí svítit. Nejde o představu úplného vyloučení člověka — i temná továrna má konstruktéry, kteří rozhodují, co se bude vyrábět — nýbrž o vyloučení člověka z opakujících se úkonů. Tato práce zkoumá, nakolik lze týž princip uplatnit ve vývoji softwaru a kde jsou jeho hranice.

1.1 Motivace

S rozšířením velkých jazykových modelů se objevila možnost automatizovat i psaní kódu. Většina dostupných nástrojů však řeší jen dílčí krok: vygenerují návrh změny, který někdo musí zasadit do procesu, ověřit a schválit. Chybí popis toho, jak má takový nástroj zapadnout do vývojového procesu jako celku — kdo schvaluje, co se stane při selhání a jak se pozná, že je výsledek správný.

Právě tato mezera je předmětem práce. Zajímavá není otázka, zda model dokáže napsat kód; to je dnes doloženo. Zajímavá je otázka, jaké okolí musí kolem takového modelu vzniknout, aby jeho výstupu bylo možné důvěřovat.

1.2 Cíl práce

Cílem této práce je navrhnout, realizovat a ověřit systém, který automatizuje vývojový proces od přijetí požadavku po vytvoření ověřené změny, aniž by se vzdal lidského schválení v rozhodujících bodech.

Dílčí cíle:

1. Popsat současný stav automatizace vývoje softwaru a orchestrace jazykových modelů.
2. Navrhnout architekturu systému, který provede požadavek celým procesem.
3. Systém realizovat a nasadit na reálné repozitáře.
4. Vyhodnotit jeho chování a pojmenovat omezení, na která v provozu narazil.

1.3 Metodika

Práce je z povahy tématu konstrukční: hlavním výstupem je funkční systém, nikoli měření. Postup odpovídá vývoji softwaru — po nastudování východisek následoval návrh, realizace a nasazení na reálné repozitáře, přičemž zjištění z provozu se vracela zpět do návrhu. Ověření proto neprobíhalo formou experimentu s kontrolní skupinou, nýbrž sledováním chování systému v provozu. Sledovány byly zejména nalezené chyby, neboť právě ty ukazují na rozdíl mezi předpokladem a skutečností.

1.4 Struktura práce

Kapitola 2 shrnuje teoretická východiska: řízení verzí, kontinuální integraci, orchestraci jazykových modelů a řízení automatizovaných změn. Kapitola 3 popisuje vlastní systém DarkFactory — jeho architekturu, životní cyklus požadavku a způsob, jímž rozpoznává obsah repozitáře. Kapitola 4 hodnotí výsledky nasazení včetně chyb, které se projevily až v provozu, a kapitola 5 je shrnuje.

2 Teoretická část

Tato kapitola vymezuje pojmy, o které se opírá praktická část: řízení verzí, kontinuální integraci, orchestraci jazykových modelů a řízení automatizovaných změn. Cílem není vyčerpávající přehled, nýbrž zavedení pojmů v podobě, v jaké s nimi pracuje navržený systém.

2.1 Řízení verzí

Systém pro řízení verzí uchovává historii změn zdrojového kódu. Distribuovaný model, jehož nejrozšířenějším zástupcem je Git, se od centralizovaného liší tím, že každý vývojář má úplnou kopii historie (1). Změny lze proto vytvářet a zkoumat i bez spojení se serverem a slučovat je až ve chvíli, kdy jsou hotové.

2.1.1 Větvě a jejich role

Větev je pojmenovaný ukazatel na určitý stav historie. Práce na nové vlastnosti probíhá ve větví oddělené od hlavní vývojové linie, takže rozpracovaný stav neovlivní ostatní. Tento postup má i důsledek pro automatizaci: dokud změna existuje pouze ve větví, lze ji libovolně ověřovat, aniž by hrozila škoda.

2.1.2 Model pull requestu

Sloučení větve do hlavní linie se ve většině dnešních projektů odehrává prostřednictvím *pull requestu* — návrhu změny, který lze komentovat, ověřovat a schvalovat. Pull request je proto přirozeným místem, kde se uplatňuje kontrola kvality, a zároveň místem, kam lze vložit schvalovací bod pro člověka.

2.2 Kontinuální integrace

Kontinuální integrace (angl. *continuous integration*) je praxe, při níž se každá změna automaticky sestaví a otestuje (2). Namísto dlouhých období, kdy se změny hromadí a slučují až na konci, se ověřuje průběžně a v malých dávkách, takže chyba je odhalena blízko svému vzniku.

2.2.1 Požadované kontroly

Ke kontinuální integraci patří pojem *požadovaných kontrol* (angl. *required checks*): množina úloh, které musí skončit úspěšně, jinak nelze změnu sloučit. Tím se z kvality stává vlastnost vynucovaná strojem, nikoli pouze dohodou mezi vývojáři.

Návrh požadovaných kontrol má jedno nezřejmé úskalí. Úloha, která se za jistých okolností „přeskočí“, nehlásí žádný výsledek; je-li přitom uvedena mezi požadovanými, zablokuje slučování napořád. Kontrola proto musí vždy skončit nějakým závěrem — i kdyby jím bylo konstatování, že v daném repozitáři není co dělat.

2.2.2 Sestavení a artefakty

Výsledkem sestavení bývá *artefakt*: spustitelný soubor, knihovna, nebo — jak ukazuje praktická část — vysázený dokument. Automatizace vydávání verzí spojuje artefakt se značkou v historii, takže ke každé vydané verzi existuje doložitelný výstup.

2.3 Jazykové modely a jejich orchestrace

Velké jazykové modely (angl. *large language models*) jsou statistické modely jazyka, které lze použít i ke generování a úpravě zdrojového kódu. Pro praktické nasazení je však podstatnější než samotná schopnost generovat text to, jak je model zasazen do procesu.

2.3.1 Kontext a nástroje

Model sám o sobě nevidí repozitář. Aby mohl provést změnu, potřebuje kontext (popis úkolu, obsah souborů) a nástroje (čtení souborů, spouštění příkazů). Kvalita výsledku závisí na obojím přinejmenším stejně jako na modelu samotném.

2.3.2 Ověřitelnost výstupu

Zásadní je, zda lze výsledek strojově ověřit. Tam, kde existují testy a statická kontrola, může systém sám rozpoznat, že se změna nepovedla, a pokusit se o nápravu. Tam, kde správnost posoudí až člověk, zůstává přínos automatizace omezený — a tato hranice je pro celý přístup určující.

2.3.3 Problém jediného poskytovatele

Spoléhá-li systém na jediného poskytovatele modelu, zastaví se ve chvíli, kdy tento poskytovatel vyčerpá přidělenou kvótu, změní rozhraní nebo je nedostupný. Řešením je vrstva, která umí tentýž úkol předat kterémukoli z několika poskytovatelů. Rozhraní se u jednotlivých poskytovatelů liší, takže tato vrstva musí úkol popsat způsobem nezávislým na konkrétním rozhraní.

2.4 Governance automatizovaných změn

Čím je systém samostatnější, tím důležitější je otázka, kde do procesu vstupuje člověk. Úplná autonomie není cílem; cílem je autonomie v rutinních krocích a lidské rozhodnutí tam, kde je nevratné nebo kde chybí měřítko správnosti.

2.4.1 Schvalovací body

Schvalovací bod je místo, kde se proces zastaví a čeká na potvrzení. Jeho umístění je kompromisem: příliš mnoho schvalování popírá smysl automatizace, příliš málo znamená ztrátu kontroly. Osvědčeným řešením je schvalovat *záměr* (co se má udělat) a *plán* (jak se to má udělat) dříve, než vznikne kód.

2.4.2 Dohledatelnost

Aby byla automatizovaná změna přezkoumatelná, musí být zřejmé, z jakého požadavku vzešla. Uchování doslovného znění původního zadání je proto součástí návrhu, nikoli formalitou: parafráze ztrácí význam, který do zadání vložil ten, kdo je formuloval.

2.4.3 Hlášení selhání

Systém, který své vlastní selhání zamlčí, je nebezpečnější než systém, který zjevně spadne: nespolehlivost je v něm neviditelná. U automatizovaného provozu je proto hlášení chyb stejně důležitou vlastností jako vlastní funkce, jak ukazují i výsledky uvedené v kapitole 4.

3 Praktická část

Praktickou částí práce je systém **DarkFactory** — sada pravidel, skriptů a pracovních postupů, které z repozitáře udělají samostatně pracující vývojový provoz. Zdrojový kód je veřejně dostupný (3).

3.1 Cíl a rozsah systému

Systém má převzít rutinní kroky vývojového procesu: přijetí požadavku, jeho interpretaci, naplánování, provedení změny a její ověření. Nemá nahradit rozhodování o tom, co se má stavět; to zůstává člověku, a systém je navržen tak, aby si toto rozhodnutí vyžádal dříve, než začne pracovat.

3.2 Architektura

Návrh vychází z jednoho požadavku: pracovní postupy i skripty musí být ve všech repozitářích totožné. Kdyby se kopírovaly, začaly by se rozcházet, a oprava chyby by se musela provádět tolikrát, kolik je repozitářů.

3.2.1 Sdílení místo kopírování

Pracovní postupy jsou proto *volané*, nikoli kopírované. Repozitář, který systém používá, obsahuje pouze krátký soubor odkazující na sdílený pracovní postup připnutý ke konkrétní verzi. Připnutí je podstatné: bez něj by se změna sdíleného postupu okamžitě promítla do všech repozitářů, včetně těch, které na ni nejsou připraveny.

3.2.2 Jediný konfigurační soubor

Vše, co se mezi repozitáři liší, je soustředěno do jediného souboru `darkfactory.json`. Ten popisuje totožnost repozitáře, jeho oblasti, nástěnky, na které patří jeho úkoly, a případné odchylky od výchozího chování. Sdílený postup je díky tomu ve všech repozitářích shodný bajt po bajtu.

3.3 Životní cyklus požadavku

Požadavek prochází systémem v pevně daných krocích, mezi nimiž jsou schvalovací body:

1. Uživatel založí požadavek s doslovným zněním svého zadání.
2. Systém požadavek interpretuje a čeká na schválení této interpretace.
3. Po schválení vzniká plán jako samostatný podřízený úkol.
4. Po schválení plánu vzniká větev a návrh změny.
5. Změna projde vlastním přezkoumáním a ověřením proti plánu.
6. Po splnění požadovaných kontrol a schválení je změna sloučena.

Doslovné znění zadání je uchováno záměrně. Zkušenost z vývoje ukázala, že právě parafráze bývá zdrojem nedorozumění: shrnutí požadavku se zdá výstižné tomu, kdo je psal, a přitom už neobsahuje to, na čem zadavateli záleželo.

3.4 Popis prostředí repozitáře

Aby systém věděl, které úlohy má spustit, musí rozpoznat, z čeho se repozitář skládá. Rozpoznávání vychází z názvů souborů popisujících balíček; jejich přítomnost je spolehlivější než jakýkoli ruční záznam, protože se nemůže rozejít se skutečností.

Soubor	Prostředí	Doména
pyproject.toml	Python	kód
Cargo.toml	Rust	kód
package.json	Node	kód
go.mod	Go	kód
typst.toml	Typst	text
.latexmkrc	LaTeX	text
lakefile.toml	Lean	matematika

Tabulka 1: Rozpoznávání prostředí podle souboru popisujícího balíček.

3.4.1 Domény a prostředí

Rozlišení dvou úrovní se ukázalo jako nutné až v průběhu práce. *Prostředí* říká, jaký nástroj je potřeba; *doména* říká, jakému způsobu řízení výsledek podléhá. Python a Rust jsou dvě prostředí téže domény — kód se testuje a balí. Typst a LaTeX jsou dvě prostředí jiné domény — text se sází a publikuje.

Toto rozlišení dovoluje popsat i repozitář, který obsahuje zároveň program a text — jako právě tato práce, jejíž praktickou částí je software popisovaný v Tabulka 1. Bez něj by bylo nutné buď považovat sazbu za zvláštní případ kódu, nebo pro texty vytvořit samostatný systém.

3.4.2 Deklarace jako doplněk rozpoznávání

Rozpoznávání nemůže vidět všechno. Repozitář této práce například přibaluje vlastní písma, takže příkaz pro sazbu není výchozí. Konfigurační soubor proto umožňuje výchozí chování přepsat, aniž by bylo nutné vypnout rozpoznávání jako celek.

3.5 Orchestrace napříč poskytovateli

Systém neváže na jednoho poskytovatele modelu. Úkol je popsán způsobem nezávislým na rozhraní a předán prvnímu dostupnému poskytovateli; vyčerpá-li tento kvótu, předá se tentýž úkol dalšímu v pořadí, místo aby se proces zastavil.

Tato vlastnost byla přímou reakcí na provozní zkušenost: zastavení celého procesu kvůli vyčerpané kvótě jediného poskytovatele bylo nejčastější příčinou prostoje.

3.6 Ověřování změn

Každá změna musí projít požadovanými kontrolami. Ty jsou navrženy tak, aby vždy skončily nějakým výsledkem — úloha, která se může „přeskočit“, by jinak zablokovala slučování napořád, jak bylo vysvětleno v kapitole 2.

```
paper:
  steps:
    - name: Detect the paper domain
      id: detect
    - name: Typeset with Typst
      if: steps.detect.outputs.typst == 'true'
    - name: No paper in this repository
      if: steps.detect.outputs.present != 'true'
      run: echo "nothing to typeset."
```

Výpis 1: Zjednodušená úloha, která vždy skončí výsledkem.

Struktura v Výpis 1 je pro celý systém typická: nejprve se zjistí, zda je v repozitáři co dělat, a teprve pak se pracuje. Poslední krok zajišťuje, že úloha skončí úspěchem i tam, kde není co ověřovat.

3.7 Hlášení selhání

Selhání kterékoli úlohy zakládá úkol s odkazem na neúspěšný běh. Opakované selhání téže úlohy nezakládá další úkol, nýbrž doplní komentář ke stávajícímu; bez toho by úloha selhávající při každé změně zahltila seznam úkolů. Jakmile úloha znovu uspěje, úkol se sám uzavře.

4 Výsledky a diskuse

4.1 Metodika ověření

Systém byl nasazen na tři repozitáře různé povahy: na vlastní repozitář systému, na aplikaci psanou v několika jazycích a na menší projekt. Sledovány byly tři veličiny: počet vlastních skriptů a pracovních postupů v jednotlivých repozitářích, počet řádků odstraněného duplicitního kódu a chování systému v provozu.

4.2 Sjednocení pracovních postupů

Sdílení jednoho pracovního postupu místo kopií vedlo k odstranění přibližně 7 800 řádků duplicitního kódu.

Repozitář	Skripty před	Skripty po
DarkFactory	12	12
omnis	7	1
ChessWithQuests	5	1

Tabulka 2: Počet vlastních skriptů před sjednocením a po něm.

Údaje v Tabulka 2 je třeba číst se dvěma výhradami. U repozitáře DarkFactory se počet neměnil, protože právě on skripty vlastní. Zbývá jeden skript v ostatních repozitářích popisuje strukturu jejich vlastní dokumentace, a proto sdílet nelze.

4.3 Rozšíření na texty

Po sjednocení byl systém rozšířen tak, aby popsal i repozitář obsahující text místo programu. Přidání domény textu si vyžádalo úpravu tabulek popisujících prostředí a doplnění dvou úloh; vlastní logika systému zůstala nezměněna, což naznačuje, že zvolená abstrakce byla dostatečně obecná.

Ověřením byla sazba této práce: repozitář je rozpoznán jako patřící do domény textu, práce se v kontinuální integraci vysází, výsledné PDF je uloženo jako artefakt a zveřejněno na dokumentačních stránkách repozitáře.

4.4 Chyby, které se projevily až v provozu

Nasazení odhalilo několik chyb, které se při návrhu neprojevily. Jsou uvedeny proto, že tvoří nejcennější část výsledků: šlo vesměs o chyby v řízení procesu, nikoli ve schopnosti změnu vytvořit.

Uvážnutí souběžnosti Volající i volaný pracovní postup použily stejný název skupiny souběžnosti, takže volaný čekal na skupinu drženou vlastním volajícím. Běh skončil bez jediné úlohy a bez chybového hlášení.

Přejmenování kontrol Volaný pracovní postup hlásí výsledek pod složeným názvem, který obsahuje i jméno volající úlohy. Ochrana větve vyžadující původní název by zablokovala každé sloučení, protože kontrola s tímto názvem už neexistuje.

Tiché selhání zápisu na nástěnce Zápisy na projektovou nástěnce byly obaleny zachycením všech výjimek, takže selhání skončilo hlášením o úspěchu. Chyba vyšla najevo až ručním porovnáním obsahu nástěnky s očekáváním.

Předpoklad o jazyce Sdílené úlohy předpokládaly, že každý repozitář obsahuje Python. Repozitář s textem proto neprošel ani prvním krokem, přestože sazba sama byla v pořádku.

4.5 Diskuse

Nejpoučnější z uvedených chyb je tiché selhání zápisu na nástěnce. Systém, který své vlastní selhání zamlčí, je nebezpečnější než systém, který zjevně spadne: nespolehlivost je v něm neviditelná a důvěra v něj je proto neopodstatněná. Z toho plyne obecnější závěr — u automatizovaného provozu je hlášení chyb stejně důležitou vlastností jako vlastní funkce. Druhým opakujícím se motivem je předpoklad o podobě repozitáře. Chyba s předpokladem o jazyce má stejnou příčinu jako potřeba zavést domény: sdílený systém musí popisovat, co v repozitáři skutečně je, a nikoli předpokládat, že se podobá tomu, pro který byl původně napsán.

4.6 Omezení

Systém předpokládá, že úkol lze ověřit strojově. Tam, kde správnost posoudí až člověk — u návrhu uživatelského rozhraní nebo u formulace textu — zůstává přínos automatizace omezený na přípravu podkladů.

Druhým omezením je závislost na dostupnosti poskytovatelů jazykových modelů. Postupné předávání úkolu mezi poskytovateli riziko snižuje, neodstraňuje je však úplně: vyčerpají-li kvótu všichni, proces se zastaví stejně jako dříve.

Třetím omezením je rozsah ověření. Systém byl nasazen na tři repozitáře jediného autora, takže zjištění nelze bez dalšího zobecnit na větší tým, kde by přibýly otázky souběžné práce více lidí nad týmž kódem.

5 Závěr

Cílem práce bylo navrhnout, realizovat a ověřit systém automatizující vývojový proces od přijetí požadavku po vytvoření ověřené změny, aniž by se vzdal lidského schválení v rozhodujících bodech. Cíl byl naplněn: systém DarkFactory byl realizován a nasazen na tři repozitáře, kde odstranil přibližně 7 800 řádků duplicitního kódu a převzal rutinní kroky vývojového procesu.

Dílčí cíle byly splněny takto. Současný stav automatizace a orchestrace jazykových modelů je popsán v kapitole 2. Architektura systému, jeho životní cyklus a způsob rozpoznávání prostředí jsou popsány v kapitole 3. Nasazení proběhlo na tři repozitáře a jeho výsledky jsou vyhodnoceny v kapitole 4.

Ukázalo se, že hlavní obtíž nespočívá v generování změn, ale v jejich řízení — v tom, kde do procesu vstupuje člověk a jak systém hlásí vlastní selhání. Všechny chyby nalezené při nasazení se týkaly této oblasti, nikoli schopnosti změnu vytvořit. Nejzávažnější z nich, tiché selhání zápisu na projektovou nástěnku, byla neviditelná právě proto, že systém hlásil úspěch.

Práce rovněž ověřila, že zvolená abstrakce je dostatečně obecná: rozšíření systému o doménu textu si vyžádalo doplnění tabulek a dvou úloh, nikoli zásah do vlastní logiky. Tato práce je toho dokladem — je sázena týmž systémem, který popisuje.

Další rozvoj se nabízí ve dvou směrech. Prvním je rozšíření o další domény, například o strojově ověřované matematické důkazy, pro něž je systém již připraven. Druhým je zpřesnění hlášení chyb tak, aby každé selhání zanechalo trvalý a viditelný záznam — v této práci byl učiněn první krok, kdy selhání zakládá úkol, který se po nápravě sám uzavře.

Seznam příloh

A Obsah přiloženého média	19
B Struktura konfiguračního souboru	20

A Obsah přiloženého média

Odevzdaný archiv obsahuje zdrojové soubory této práce a odkaz na veřejný repozitář se zdrojovým kódem systému DarkFactory.

B Struktura konfiguračního souboru

Ukázka konfigurace popisující jeden repozitář sdílenému pracovnímu postupu.

Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
2. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
3. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>